

得分	评阅人



华中农业大学
HUAZHONG AGRICULTURAL UNIVERSITY

基于线性规划和遗传算法 实现生猪饲养配方模型优化

课程名称 : 算法程序与设计

指导教师 : 朱荣波

成 员 : 李莎 张高博 孙华斌 王绮文

学 号 : 2021306220119

班 级 : 计算机 2208 班

学 院 : 信息学院

目录

1. 实验目的:	1
2. 实现思路	1
2.1. 饲料配方的评判标准	1
2.2. 实验数据来源	1
2.3. 多约束条件的处理	2
3. 枚举搜索	3
3.1. 原理解释:	3
3.2. 数据模型:	3
3.3. 核心代码:	3
3.3.1. 数据初始化	3
3.3.2. 循环尝试不同组合并判断是否满足条件	5
3.3.3. 记录最优组合	5
3.4. 代码运行展示:	5
4. 2.线性规划法	9
4.1. 原理解释:	9
4.2. 数据模型:	9
4.3. 代码展示:	10
4.3.1. 数据初始化	10
4.3.2. 创建决策变量	11
4.3.3. 目标函数	11
4.3.4. 约束条件	12
4.4. 代码运行展示:	13
5. 3 遗传算法	16
5.1. 原理解释:	16
5.1.1. 初始种群产生	16
5.1.2. 适应度函数	17
5.1.3. 选择操作	17
5.1.4. 交叉操作	17
5.1.5. 变异	18
5.2. 数据模型:	18
5.3. 代码分析:	19
5.3.1. 决策变量的初始化	19
5.3.2. 约束条件	19
5.3.3. 对与交叉阶段的约束条件	20
5.3.4. 对于变异阶段的约束条件	21
5.3.5. 进化函数	22
5.3.6. 决策过程展示	23
5.4. 代码运行展示:	24
6. 不同算法的性能分析	28
6.1. 数据对比	28
6.2. 模型评价	29
7. 参考文献	31

8. 附录.....	31
9. 小组分工贡献:	32

1. 实验目的:

应用计算机技术、运筹学及线性规划方法,通过大数据设计配合饲料配方时,将动物对各营养素的最适需要量和饲料原料的营养成分及价格作为已知条件,把满足动物营养需要量作为约束条件,再把最低饲料成本作为配方设计的目标函数。这种方法也是目前应用最广泛的饲料配方设计方法。

基于线性规划法、目标规划法、遗传算法三者之一建立生猪饲料配方模型,并在实验数据集上给出结果。

2. 实现思路

2.1. 饲料配方的评判标准

饲料配方设计追求营养目标和经济目标。

营养目标则需从两个方面进行衡量,一是饲料配方的营养成分是否达到饲养标准的要求,二是饲料配方中各营养成分之间是否达到平衡。也就是说猪饲料配方的营养成分含量必须达到饲养标准的要求,才能满足猪的营养需求。

经济目标可用饲料配方成本衡量,饲料配方成本越低则经济效益越高。

2.2. 实验数据来源

算法以猪饲料配方为研究对象,设饲料总重量为 1kg。饲料原料及其营养成分表见附录 1。

2.3. 多约束条件的处理

根据附录，猪饲料配方中共涉及 13 种原料，14 种营养要求，除 2 种原料为等量使用，其余皆为约束，所以共计 25 个约束条件。

在本文的遗传算法中设计了配方测试函数，测试函数有两个功能：功能一，测试配方中除等量使用的原料外，其他 11 种原料是否满足用量的上、下限要求；功能二，满足功能一后，测试该组成配方的 14 种营养是否满足本阶段猪生长的需求。

配方测试函数在每次新生成配方时进行调用，如果新配方不能通过测试，则重复生成新配方的操作。配方测试函数在初始种群生成、交叉和变异这三个产生新配方的操作中调用，从而保证了新配方（种群）完全满足 25 个约束条件。

3. 枚举搜索

3.1. 原理解释:

基于营养标准和使用比例的条件，通过遍历不同的饲料组合，找到最优的组合使得生猪在成本最小的情况下满足其营养需求。

3.2. 数据模型:

使用了多层嵌套的循环，对一系列参数进行组合，然后通过一系列条件判断，筛选出满足条件的组合。整体而言，它在搜索一组参数的空间中，找到那些满足一定条件的组合。

1. 构建循环结构枚举数值
2. 条件判断是否满足范围条件
3. 输出每一种方案的数量
4. 将最优方案记录下来

3.3. 核心代码:

3.3.1. 数据初始化

`get_standard` 函数： 从名为"生猪饲养标准"的 Excel 表中读取数据，参数 `lie` 表示所需数据的列，返回该列的数据。

`get_price` 函数： 从名为"饲料原料及其营养成分含量"的 Excel 表中读取数据，提取第 18 行（索引为 17，因为 Python 中索引从 0 开始）除第一个元素外的数据，即各种原料的价格。

`get_data` 函数： 从名为"饲料原料及其营养成分含量"的 Excel 表中读取数据，提取各种原料的数据并返回。

```

def get_standard(lie):
    df=pd.read_excel(r"2023秋算法大作业"
                    r"数据.xlsx",sheet_name='生猪饲养标准')
    return df[lie].values

1 usage
def get_price():
    df = pd.read_excel(r"2023秋算法大作业数据.xlsx", sheet_name='饲料原料及其营养成分含量')
    price = df.iloc[17, 1:].values
    return price

def get_data():
    df=pd.read_excel(r"2023秋算法大作业数据.xlsx",sheet_name='饲料原料及其营养成分含量')
    yumi_data = df['玉米']
    yumi_data = yumi_data

    maifu_data = df['麦麸']
    maifu_data = maifu_data

    doupo_data = df['豆粕']
    doupo_data = doupo_data

    yufen_data = df['鱼粉']
    yufen_data = yufen_data

    mianziyou_data = df['棉籽油']
    mianziyou_data = mianziyou_data

    caiziyou_data = df['菜籽油']
    caiziyou_data = caiziyou_data

    # 每个原料取前14个(到"总磷")
    # yumi_data[:14]
    yumi_data,maifu_data,doupo_data,yufen_data,mianziy

    yumi_data=yumi_data[:14]
    maifu_data = maifu_data[:14]
    doupo_data = doupo_data[:14]
    yufen_data = yufen_data[:14]
    mianziyou_data = mianziyou_data[:14]
    caiziyou_data = caiziyou_data[:14]
    zhiwuyou_data = zhiwuyou_data[:14]
    shifen_data = shifen_data[:14]
    linsuanqinggai_data = linsuanqinggai_data[:14]
    danannsu_data = danannsu_data[:14]
    laiannsu_data = laiannsu_data[:14]

```

3.3.2. 循环尝试不同组合并判断是否满足条件

使用嵌套循环，每个循环对应一个参数的取值范围，遍历各种可能的饲料组合，每个原料的比例在一定范围内变化。

在最内层的循环内，有一系列条件判断，用于筛选符合特定条件的参数组合。对每个组合进行条件判断，确保满足生猪的营养标准和各个原料的使用比例条件。

```
for yumi in np.arange(0.2,1.2,0.1):
    for maifu in np.arange(0, 0.5, 0.1):
        for doupo in np.arange(0.1, 1.2, 0.1):
            for yufen in np.arange(0, 0.2, 0.1):
                for mianziyou in np.arange(0, 0.4, 0.1):
                    for caiziyou in np.arange(0, 0.5, 0.1):
                        for zhiwuyou in np.arange(0,0.2,0.1):
                            for shifen in np.arange(0,0.1,0.1):
                                for linsuanqinggai in np.arange(0,0.3,0.1):
                                    for danannsu in np.arange(0,0.2,0.1):
                                        for laiannsu in np.arange(0,0.2,0.1):
                                            # f/(f+s+m)=0.01 同时 s/(f+s+m)=0.003 目的求f和s关于m的表达式
                                            # f=(997*3m)/(997*99+3)    s=100m/(997*33-1)
                                            fuhesyunhunliao=997*3*(yumi + maifu + doupo + yufen + mianziyou + caiziyou)/(997*99+3)
                                            shiyansuan=100*(yumi + maifu + doupo + yufen + mianziyou + caiziyou)/(997*33-1)
                                            if (yumi * yumi_data + maifu * maifu_data + doupo * doupo_data + yufen * yufen_data + mianziyou * mianziyou_data + caiziyou * caizi
                                                and 0.2<=(yumi)/(yumi+maifu+doupo+yufen+mianziyou+caiziyou)<=0.9\
                                                and 0<=(maifu)/(yumi+maifu+doupo+yufen+mianziyou+caiziyou)<=0.2\
                                                and 0.1<=(doupo)/(yumi+maifu+doupo+yufen+mianziyou+caiziyou)<=0.6\
                                                and 0<=(yufen)/(yumi+maifu+doupo+yufen+mianziyou+caiziyou)<=0.05\
                                                and 0<=(mianziyou)/(yumi+maifu+doupo+yufen+mianziyou+caiziyou)<=0.09\
                                                and 0<=(caiziyou)/(yumi+maifu+doupo+yufen+mianziyou+caiziyou)<=0.07:
                                                    count+=1
                                                    print("方案{}:".format(count))
                                                    prices=sum(price * [yumi, maifu, doupo, yufen, mianziyou, caiziyou, zhiwuyou, shifen,
                                                        linsuanqinggai, danannsu, laiannsu, fuhesyunhunliao, shiyansuan])
                                                    print("总价格: {} 元".format(prices))
                                                    print("用量:玉米:{} kg 麦麸:{} kg 豆粕:{} kg 鱼粉:{} kg \n"
```

3.3.3. 记录最优组合

如果是最优组合，记录该组合的价格和用量。使用 NumPy 的数组进行存储。

```
yumi.round(3), maifu.round(3), doupo.round(3), yufen.round(3), mianziyou.round(3), caiziyou.round(3), zhiwuyou.round(3),
shifen.round(3), linsuanqinggai.round(3), danannsu.round(3), laiannsu.round(3), fuhesyunhunliao.round(3), shiyansuan.round(3)))

new_row = np.array([yumi.round(3), maifu.round(3), doupo.round(3), yufen.round(3),
                    mianziyou.round(3), caiziyou.round(3), zhiwuyou.round(3),
                    shifen.round(3), linsuanqinggai.round(3), danannsu.round(3),
                    laiannsu.round(3), fuhesyunhunliao.round(3),
                    shiyansuan.round(3)])
numCount=np.append(numCount,[new_row],axis=0)
sumNumPrice=np.append(sumNumPrice,prices)
print('\n')
```

3.4. 代码运行展示:

1.对于 20-50kg 的生猪，每一千克需要的饲料结果如下:

请输入生猪的重量(20-120kg):40

方案1:

总价格: 10.817242025487621 元

用量:玉米:0.4 kg 麦麸:0.4 kg 豆粕:1.0 kg 鱼粉:0.1 kg

棉籽油:0.0 kg 菜籽油:0.1 kg 植物油:0.1 kg 石粉:0.0 kg

磷酸氢钙:0.2 kg 蛋氨酸:0.1 kg 赖氨酸:0.1 kg 复合预混料:0.02 kg 食盐:0.006 kg

总数量:819

最优方案为: 方案757

最优价格为: 9.907242025487625

最优用量:玉米:1.1 kg 麦麸:0.4 kg 豆粕:0.3 kg 鱼粉:0.1 kg

棉籽油:0.0 kg 菜籽油:0.1 kg 植物油:0.1 kg 石粉:0.0 kg

磷酸氢钙:0.2 kg 蛋氨酸:0.1 kg 赖氨酸:0.1 kg 复合预混料:0.02 kg 食盐:0.006 kg

最优目标函数值为 10.81724205487621, 代表最佳的情况下 1kg

生猪饲料需要约 10.817 元

当取得最优函数值时每 1 千克饲料所需各种原料进取值如下表

原料种类	质量 (kg)
玉米	1.1
麦麸	0.4
豆粕	0.3
鱼粉	0.1
棉籽油	0.0
菜籽油	0.1
植物油	0.1
石粉	0.0
磷酸氢钙	0.2
蛋氨酸	0.1
赖氨酸	0.1
复合预混料	0.02
食盐	0.006

2.对于 50-80kg 的生猪, 每一千克需要的饲料结果如下:

请输入生猪的重量(20-120kg):60

方案1:

总价格: 9.896155721664481 元

用量:玉米:0.4 kg 麦麸:0.1 kg 豆粕:1.0 kg 鱼粉:0.0 kg

棉籽油:0.1 kg 菜籽油:0.1 kg 植物油:0.1 kg 石粉:0.0 kg

磷酸氢钙:0.2 kg 蛋氨酸:0.1 kg 赖氨酸:0.1 kg 复合预混料:0.017 kg 食盐:0.005 kg

总数量:1115

最优方案为: 方案2

最优价格为: 9.88615572166448

最优用量:玉米:0.4 kg 麦麸:0.2 kg 豆粕:1.0 kg 鱼粉:0.0 kg

棉籽油:0.0 kg 菜籽油:0.1 kg 植物油:0.1 kg 石粉:0.0 kg

磷酸氢钙:0.2 kg 蛋氨酸:0.1 kg 赖氨酸:0.1 kg 复合预混料:0.017 kg 食盐:0.005 kg

最优目标函数值为 9.88615572166448, 代表最佳的情况下 1kg 生

猪饲料需要约 9.886 元

当取得最优函数值时每 1 千克饲料所需各种原料进取值如下表

原料种类	质量 (kg)
玉米	0.4
麦麸	0.2
豆粕	1.0
鱼粉	0.0
棉籽油	0.0
菜籽油	0.1
植物油	0.1
石粉	0.0
磷酸氢钙	0.2
蛋氨酸	0.1
赖氨酸	0.1
复合预混料	0.017
食盐	0.00

3.对于 80-120kg 的生猪, 每一千克需要的饲料结果如下:

请输入生猪的重量(20-120kg):100

方案1:

总价格: 9.445431519115719 元

用量:玉米:0.3 kg 麦麸:0.1 kg 豆粕:0.9 kg 鱼粉:0.0 kg

棉籽油:0.1 kg 菜籽油:0.1 kg 植物油:0.1 kg 石粉:0.0 kg

磷酸氢钙:0.2 kg 蛋氨酸:0.1 kg 赖氨酸:0.1 kg 复合预混料:0.015 kg 食盐:0.005 kg

总数量:1314

最优方案为: 方案2

最优价格为: 9.315431519115716

最优用量:玉米:0.3 kg 麦麸:0.2 kg 豆粕:0.8 kg 鱼粉:0.0 kg

棉籽油:0.1 kg 菜籽油:0.1 kg 植物油:0.1 kg 石粉:0.0 kg

磷酸氢钙:0.2 kg 蛋氨酸:0.1 kg 赖氨酸:0.1 kg 复合预混料:0.015 kg 食盐:0.005 kg

最优目标函数值为 9.315431519115716, 代表最佳的情况下 1kg

生猪饲料需要约 9.315 元

当取得最优函数值时每 1 千克饲料所需各种原料进取值如下表

原料种类	质量 (kg)
玉米	0.3
麦麸	0.2
豆粕	0.8
鱼粉	0.0
棉籽油	0.1
菜籽油	0.1
植物油	0.1
石粉	0.0
磷酸氢钙	0.2
蛋氨酸	0.1
赖氨酸	0.1
复合预混料	0.01
食盐	0.005

4.2.线性规划法

应用场景：给出一个阶段中猪所需的各营养成分含量、各原料所含营养成分，建立模型，输出各原料饲喂量，满足各营养成分需求。

4.1. 原理解释：

线性规划在确定决策变量、目标函数和约束条件后，用线性规划约束函数可以很方便的对相关问题进行优化求解。线性规划用于求解生猪饲料配方时，是将饲料配方中的相关因素和约束条件转化为线性数学函数，在一定的约束条件下求出最小值。由于饲料配方中含有众多的营养成分，不同体重阶段的生猪饲养标准也不同，因此在饲料配比时所用原料营养含量高低不同、价格各异。利用线性规划法求解出的饲料配方，既提高生猪最大生产性能，又降低饲料耗用量。

该算法有以下过程：

1. 目标函数：
2. 约束条件确定

4.2. 数据模型：

变量介绍： c_i 是每种饲料的价格，

x_i 是决策变量表示每种饲料的含量，

s_{ij} 是第 i 种饲料第 j 种营养元素的含量，

F 表示对应阶段猪所需的最小营养价值含量

最后得出每一克小猪需要的饲料配方和价格。

目标函数：

$$\text{Min } Z = \sum_{i=1}^{12} c_i x_i \quad (\text{求最小成本})$$

约束条件：

$$\text{St.} \left\{ \begin{array}{l} 20 \leq x_1 \leq 90 \\ 0 \leq x_2 \leq 20 \\ 10 \leq x_3 \leq 60 \\ 0 \leq x_4 \leq 5 \\ 0 \leq x_5 \leq 9 \\ 0 \leq x_6 \leq 7 \\ 0 \leq x_7 \leq 3 \\ 0 \leq x_8 \leq 2 \\ 0 \leq x_9 \leq 10 \\ 3 \leq x_{10} \leq 3 \\ 0 \leq x_{11} \leq 3 \\ x_{12} = 1 \\ x_{13} = 0.3 \\ \sum_{i=1}^{12} \sum_{j=1}^{13} x_j * s_{ij} \geq F_{ij} \end{array} \right.$$

4.3. 代码展示：

4.3.1. 数据初始化

`get_upperAndDown()` 函数该函数用于获取饲料原料及其营养成分含量的上下限。

`get_price()` 函数，该函数用于获取饲料原料及其营养成分含量对应的价格。

获取用户输入生猪的重量并调用相应的 `get_standard` 函数获取相应范围的饲养标准。

```

def get_standard(lie):
    df=pd.read_excel(r"2023秋算法大作业数据.xlsx",sheet_name='生猪饲养标准')
    return df[lie].values

1 usage
def get_upperAndDown():
    df = pd.read_excel(r"2023秋算法大作业数据.xlsx", sheet_name='饲料原料及其营养成分含量')
    upper = (df.iloc[14, 1:].values)/100
    down = (df.iloc[15, 1:].values)/100
    return upper,down

1 usage
def get_price():
    df = pd.read_excel(r"2023秋算法大作业数据.xlsx", sheet_name='饲料原料及其营养成分含量')
    price = df.iloc[17, 1:].values
    return price

weight = input("请输入生猪的重量(20-120kg):")
weight = float(weight)
if 20<=weight<=50:
    standard = get_standard('20-50kg')
elif 50<weight<=80:
    standard = get_standard('50-80kg')
elif 80<weight<=120:
    standard = get_standard('80-120kg')
else:
    print("请输入正确的重量!")
    exit()

```

upper 和 down 分别存储了各个决策变量的上下界，price 存储了各个决策变量对应的价格。

```

# 获取表中的上下界,已经去掉%
upper,down = get_upperAndDown()

# 获取表中的价格
price=get_price()

```

4.3.2. 创建决策变量

使用 `cp.Variable(13, pos=True)` 创建了一个包含 13 个非负实数的决策变量 `x`。

```

# 创建13个决策变量
x = cp.Variable(13, pos=True)

```

4.3.3. 目标函数

使用 `cp.Minimize` 定义了一个最小化目标函数。目标函数是各个决策变量的价格乘以相应的权重的和。

```
#目标函数
obj=cp.Minimize(price[0]*x[0]+price[1]*x[1]+
                price[2]*x[2]+price[3]*x[3]+price[4]*x[4]+price[5]*x[5]
                +price[6]*x[6]+price[7]*x[7]+price[8]*x[8]+price[9]*x[9]+price[10]*x[10]+price[11]*x[11]+price[12]*x[12])
```

4.3.4. 约束条件

这部分代码添加了线性规划问题的约束条件，包括饲料原料的上下限、各种营养成分的需求等。约束条件通过列表 `cons` 存储。第一组约束是 `x[0]` 的上下界约束。

饲料原料约束按照模式设置为 `lower_bound <= variable <= upper_bound` 的形式。

```
#约束条件
cons = [x[0] >= down[0] * sum(x[0:]), x[0] <= upper[0] * sum(x[0:]),
        x[1] >= down[1], x[1] <= upper[1] * sum(x[0:]),
        down[2] * sum(x[0:]) <= x[2], x[2] <= upper[2] * sum(x[0:]),
        down[3] <= x[3], x[3] <= upper[3] * sum(x[0:]),
        down[4] <= x[4], x[4] <= upper[4] * sum(x[0:]),
        down[5] <= x[5], x[5] <= upper[5] * sum(x[0:]),
        down[6] <= x[6], x[6] <= upper[6] * sum(x[0:]),
        down[7] <= x[7], x[7] <= upper[7] * sum(x[0:]),
        down[8] <= x[8], x[8] <= upper[8] * sum(x[0:]),
        down[9] <= x[9], x[9] <= upper[9] * sum(x[0:]),
        down[10] <= x[10], x[10] <= upper[10] * sum(x[0:]),
        x[11] == 0.01 * sum(x[0:]), x[12] == 0.003 * sum(x[0:]),
        sum(x[0:]) >= 1,
```

营养成分约束


```

3.85*x[0]+3.23*x[1]+4.05*x[2]+4.25*x[3]+3.4*x[4]+3.44*x[5]+7.7*x[6]==standard[0]*sum(x[0:]),

9.4*x[0]+17.3*x[1]+49.9*x[2]+68.5*x[3]+44.9*x[4]+37.8*x[5]>=standard[1]*sum(x[0:]),

0.43*x[0]+1.18*x[1]+3.68*x[2]+3.99*x[3]+4.96*x[4]+2.65*x[5]>=standard[2]*sum(x[0:]),

0.29*x[0]+0.49*x[1]+1.38*x[2]+1.94*x[3]+1.27*x[4]+1.06*x[5]>=standard[3]*sum(x[0:]),

1.05*x[0]+1.07*x[1]+3.9*x[2]+4.95*x[3]+2.64*x[4]+2.56*x[5]>=standard[4]*sum(x[0:]),

0.31*x[0]+0.54*x[1]+2.28*x[2]+2.8*x[3]+1.39*x[4]+1.45*x[5]>=standard[5]*sum(x[0:]),

0.2*x[0]+0.27*x[1]+0.72*x[2]+1.92*x[3]+0.71*x[4]+0.71*x[5]+99*x[9]>=standard[6]*sum(x[0:]),

0.27*x[0]+0.7*x[1]+3.13*x[2]+5.24*x[3]+1.85*x[4]+2.12*x[5]+78.8*x[10]>=standard[7]*sum(x[0:]),

0.33*x[0]+0.56*x[1]+1.99*x[2]+2.88*x[3]+1.45*x[4]+1.67*x[5]>=standard[8]*sum(x[0:]),

0.07*x[0]+0.24*x[1]+0.63*x[2]+0.72*x[3]+0.54*x[4]+0.55*x[5]>=standard[9]*sum(x[0:]),

0.43*x[0]+0.69*x[1]+2.62*x[2]+2.73*x[3]+2.38*x[4]+1.53*x[5]>=standard[10]*sum(x[0:]),

0.38*x[0]+0.78*x[1]+2.34*x[2]+3.3*x[3]+1.9*x[4]+1.79*x[5]>=standard[11]*sum(x[0:]),

0.04*x[0]+0.13*x[1]+0.4*x[2]+5.34*x[3]+0.2*x[4]+0.75*x[5]+35*x[7]+24*x[8]>=standard[12]*sum(x[0:]),

0.3*x[0]+0.18*x[1]+0.71*x[2]+3.05*x[3]+1.15*x[4]+1.1*x[5]+10*x[8]>=standard[13]*sum(x[0:])
]

```

4.4. 代码运行展示:

1.对于 20-50kg 的生猪，每一千克需要的饲料结果如下:

请输入生猪的重量(20-120kg):40

最小成本: 1.6978452256246281 元

最优用量:玉米:0.4681 kg 麦麸:0.2 kg 豆粕:0.1 kg 鱼粉:0.0 kg

棉籽油:0.09 kg 菜籽油:0.07 kg 植物油:0.0 kg 石粉:0.02 kg

磷酸氢钙:0.03819 kg 蛋氨酸:0.0 kg 赖氨酸:0.00071 kg 复合预混料:0.01 kg 食盐:0.003 kg

进程已结束,退出代码为 0

最优目标函数值为 1.69784522,代表最佳的情况下 1kg 生猪饲料需要约 1.69784522 元

当取得最优函数值时每 1 千克饲料所需各种原料进取值如下表

原料种类	质量 (kg)
玉米	0.4681
麦麸	0.2
豆粕	0.1
鱼粉	0.0

棉籽油	0.09
菜籽油	0.07
植物油	0.0
石粉	0.02
磷酸氢钙	0.03819
蛋氨酸	0.0
赖氨酸	0.0071
复合预混料	0.01
食盐	0.003

2.对于 50-80kg 的生猪，每一千克需要的饲料结果如下：

请输入生猪的重量(20-120kg):60

最小成本: 1.6727225215569348 元

最优用量:玉米:0.54717 kg 麦麸:0.2 kg 豆粕:0.1 kg 鱼粉:0.0 kg

棉籽油:0.00047 kg 菜籽油:0.07 kg 植物油:0.0 kg 石粉:0.02 kg

磷酸氢钙:0.04936 kg 蛋氨酸:0.0 kg 赖氨酸:0.0 kg 复合预混料:0.01 kg 食盐:0.003 kg

进程已结束，退出代码为 0

最优目标函数值为 1.6727225，代表最佳的情况下 1kg 生猪饲料需要约 1.6727225 元

当取得最优函数值时每 1 千克饲料所需各种原料进取值如下表

原料种类	质量 (kg)
玉米	0.54717
麦麸	0.2
豆粕	0.1
鱼粉	0.0
棉籽油	0.00047
菜籽油	0.07
植物油	0.0
石粉	0.02
磷酸氢钙	0.04936
蛋氨酸	0.0
赖氨酸	0.0
复合预混料	0.01
食盐	0.003

3.对于 80-120kg 的生猪，每一千克需要的饲料结果如下：

请输入生猪的重量(20-120kg):100

最小成本: 1.6678364935064938 元

最优用量:玉米:0.71538 kg 麦麸:0.0 kg 豆粕:0.1 kg 鱼粉:0.0 kg

棉籽油:0.0 kg 菜籽油:0.07 kg 植物油:0.0 kg 石粉:0.02 kg

磷酸氢钙:0.08162 kg 蛋氨酸:0.0 kg 赖氨酸:0.0 kg 复合预混料:0.01 kg 食盐:0.003 kg

最优目标函数值为 1.667836, 代表最佳的情况下 1kg 生猪饲料需

要约 1.667836 元

当取得最优函数值时每 1 千克饲料所需各种原料进取值如下表

原料种类	质量 (kg)
玉米	0.71538
麦麸	0.0
豆粕	0.1
鱼粉	0.0
棉籽油	0.0
菜籽油	0.07
植物油	0.0
石粉	0.02
磷酸氢钙	0.08162
蛋氨酸	0.0
赖氨酸	0.0
复合预混料	0.01
食盐	0.003

5.3 遗传算法

5.1. 原理解释：

遗传算法是一种基于生物进化过程的优化算法，模拟自然界中遗传和进化的过程，通过对候选解的适应度进行评估、选择、交叉和变异等操作来获得更优的解。具体来说，遗传算法将问题的解表示成一组染色体（即基因串），每个基因代表一个决策变量的取值，通过不断迭代选择、交叉和变异操作，使得适应度更高的基因串被保存下来，并作为下一代的“父代”参与后续的操作，从而不断地寻找到更优的解。

由于饲料配比问题是一个比较复杂，变量个数较多，约束条件比较多，精度要求较高的寻找最优解的过程，饲料配方的解空间又是连续的，故采用实数编码，即采用决策变量的真实值来进行编码。以饲料原料的使用量作为决策变量，就可以通过饲料原料用量的真实值来编码。采用实数编码，应满足在给定饲料原料的用量上下限范围内进行。用 13 个实数变量分别表示玉米、小麦、豆粕、鱼粉、棉籽粕、油菜籽粕、植物油、石粉、磷酸氢钙、蛋氨酸、赖氨酸、复合预混料、食盐（其中复合预混料、食盐是定量使用）在配合饲料中的百分含量。

5.1.1. 初始种群产生

本文采用第一种策略，根据数据饲料原料及其营养成分表中各种原料的用量上限、用量下限、等量使用的要求，在最优解所占问题空间中的分布范围内使用随机生成初始种群。

具体实现如下：1.生成随机种子；

2.设定初始种群的数量；

3.利用约束条件对生成的每一个随机数的上下限进行控制，保证生成的随机数在约束条件的范围内；

4.测试生成的种群中各原料是否能满足猪 50-80kg 这一生长阶段的营养需求，若不能满足则返回 3;5.初始种群的数量是否达到，若达到则跳出，否则返回 3。

5.1.2. 适应度函数

$y = \sum_{i=1}^{13} c_i x_i$ 其中 c_i 表示第 i 中饲料的市场价格， x_i 表示第 i 种饲料

在配方中的含量， y 为成本价格。

5.1.3. 选择操作

选择操作以个体的适应度为基础。个体适应度较高的个体被遗传到下一代群体中的概率较大。选择操作的主要目的是提高全局的收敛性和计算效率。

单个个体适应度值倒数： $\frac{1}{Y} = y = \sum_{i=1}^{13} c_i x_i$

适应度值倒数之和： $f = \sum_{j=1}^{n_p} Y_j$

选择概率： $p_j = \frac{Y_j}{f} (1 \leq j \leq n_p)$

5.1.4. 交叉操作

算法采用算数交叉。

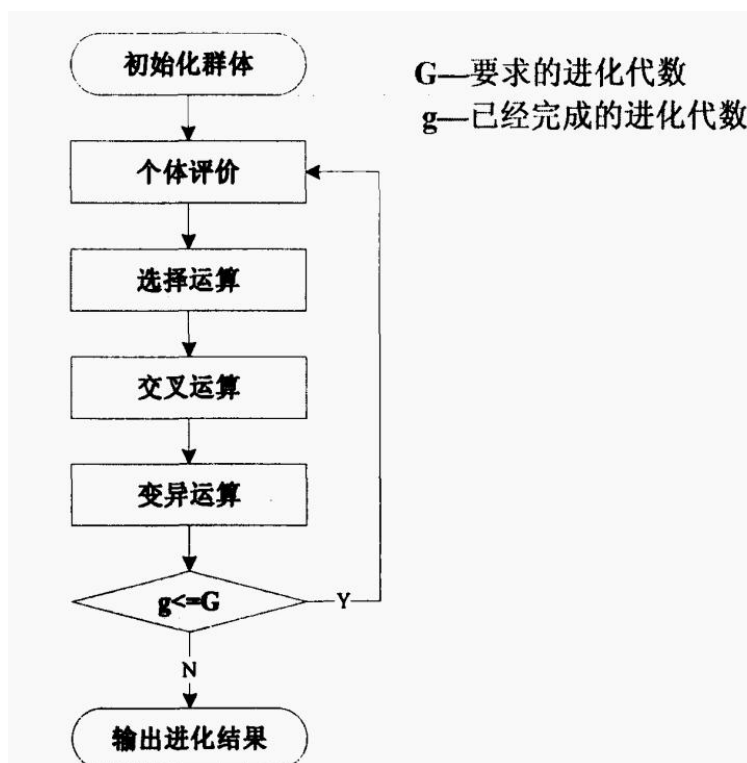
算术交叉表达式为：

$$\begin{cases} \text{child1} = \text{rnd} * \text{parent1} + (1 - \text{rnd}) * \text{parent2} \\ \text{child2} = \text{rnd} * \text{parent2} + (1 - \text{rnd}) * \text{parent1} \end{cases} (0 \leq \text{rnd} \leq 1)$$

其中 rnd 表示 $[0,1]$ 之间的随机数。

5.1.5. 变异

算法实现采用均匀变异 在变异过程中个体中的一个随机基因在约束条件的上下范围内实现随机生成并替换原有基因值。



5.2. 数据模型：

创建一个种群，实例化一个问题，规定每个因子的上下边界为相应成分的用量上限和用量下限。

我们要计算目标函数值（即最终价格），将计算结果作为种群的 objv 属性，优化条件为 objv 的值尽可能小。

设立营养量约束，按照数据的要求创建约束条件。

在本例中，我们设置的终止条件为进行 3000 轮迭代。种群规模设置为 1580.变异概率为 0.1，交叉互换率为 0.7。

十三种原料对应着成十三个因子。我们将这十三个因子组成一个十三维的向量。假设当前有 n 组配方方案，此时程序会形成 $n*13$ 的一个矩阵。每一行代表一种原料，每一列代表该原料在每种配方下所占的比例。

根据猪的不同体重还有对应的营养需求所设定的每种原料有着原料上限和下限。

最后调用算法模板，得到问题的答案。

5.3. 代码分析：

5.3.1. 决策变量的初始化

确定决策变量的上下界

```
name = 'MyProblem'

M = 1 # 初始化M (目标维数)
maxormins = [1] # 初始化目标最小最大化标记列表, 1: min; -1: max
Dim = 11 # 初始化Dim (决策变量维数)
varTypes = [0] * Dim # 初始化决策变量类型, 0: 连续; 1: 离散
lb = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] # 决策变量下界
ub = [1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000] # 决策变量上界
# 决定能否取等
lbin = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1] # 决策变量下边界
ubin = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1] # 决策变量上边界
# 调用父类构造方法完成实例化
ea.Problem.__init__(self, name, M, maxormins, Dim, varTypes, lb,
                    ub, lbin, ubin)
```

5.3.2. 约束条件

对选择阶段的约束以及所有成长阶段都要遵守的用量约束

```
(B1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.90,
(C1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.20,
0.10 - (D1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)),
(D1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.60,
(E1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.05,
(F1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.09,
(G1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.07,
(H1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,
(I1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.02,
(J1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.10,
(K1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,
(L1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,
-(
    3.85 * B1 + 3.23 * C1 + 4.05 * D1 + 4.25 * E1 + 3.4 * F1 + 3.44 * G1 + 7.7 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1)
-(
    9.4 * B1 + 17.3 * C1 + 49.9 * D1 + 68.5 * E1 + 44.9 * F1 + 37.8 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 18,
-(
    0.43 * B1 + 1.18 * C1 + 3.68 * D1 + 3.99 * E1 + 4.96 * F1 + 2.65 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.37,
-(
    0.29 * B1 + 0.49 * C1 + 1.38 * D1 + 1.94 * E1 + 1.27 * F1 + 1.06 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.3,
-(
    1.05 * B1 + 1.07 * C1 + 3.9 * D1 + 4.95 * E1 + 2.64 * F1 + 2.56 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.9,
-(
    0.31 * B1 + 0.54 * C1 + 2.28 * D1 + 2.8 * E1 + 1.39 * F1 + 1.45 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.51,
-(
    0.2 * B1 + 0.27 * C1 + 0.72 * D1 + 1.92 * E1 + 0.71 * F1 + 0.71 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 99 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.25,
-(
    0.27 * B1 + 0.7 * C1 + 3.13 * D1 + 5.24 * E1 + 1.85 * F1 + 2.12 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 78.8 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.95,
-(
    0.33 * B1 + 0.56 * C1 + 1.99 * D1 + 2.88 * E1 + 1.45 * F1 + 1.67 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.61,
-(
    0.07 * B1 + 0.24 * C1 + 0.63 * D1 + 0.72 * E1 + 0.54 * F1 + 0.55 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.17,
-(
    0.43 * B1 + 0.69 * C1 + 2.62 * D1 + 2.73 * E1 + 2.38 * F1 + 1.53 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.55,
-(
    0.38 * B1 + 0.78 * C1 + 2.34 * D1 + 3.3 * E1 + 1.9 * F1 + 1.79 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.64,
-(
    0.04 * B1 + 0.13 * C1 + 0.4 * D1 + 5.34 * E1 + 0.2 * F1 + 0.75 * G1 + 0 * H1 + 35 * I1 + 24 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.6,
-(
    0.3 * B1 + 0.18 * C1 + 0.71 * D1 + 3.05 * E1 + 1.15 * F1 + 1.1 * G1 + 0 * H1 + 0 * I1 + 10 * J1 + 0 * K1 + 0 * L1) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.5])
```

对于R阶段的约束

5.3.3. 对与交叉阶段的约束条件




```

pop.CV = np.hstack([
    0.20 - (B1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)),
    (B1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.90,
    (C1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.20,
    0.10 - (D1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)),
    (D1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.60,
    (E1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.05,
    (F1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.09,
    (G1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.07,
    (H1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,
    (I1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.02,
    (J1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.10,
    (K1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,
    (L1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,

    -(
        3.85 * B1 + 3.23 * C1 + 4.05 * D1 + 4.25 * E1 + 3.4 * F1 + 3.44 * G1 + 7.7 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ),
    -(
        9.4 * B1 + 17.3 * C1 + 49.9 * D1 + 68.5 * E1 + 44.9 * F1 + 37.8 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 15.5,
    -(
        0.43 * B1 + 1.18 * C1 + 3.68 * D1 + 3.99 * E1 + 4.96 * F1 + 2.65 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ),
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.27,
    -(
        0.29 * B1 + 0.49 * C1 + 1.38 * D1 + 1.94 * E1 + 1.27 * F1 + 1.06 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ),
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.24,
    -(
        1.05 * B1 + 1.07 * C1 + 3.9 * D1 + 4.95 * E1 + 2.64 * F1 + 2.56 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.71,
    -(
        0.31 * B1 + 0.54 * C1 + 2.28 * D1 + 2.8 * E1 + 1.39 * F1 + 1.45 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.42,
    -(
        0.2 * B1 + 0.27 * C1 + 0.72 * D1 + 1.92 * E1 + 0.71 * F1 + 0.71 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 99 * K1 + 0 * L1
    ),
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.2,
    -(
        0.27 * B1 + 0.7 * C1 + 3.13 * D1 + 5.24 * E1 + 1.85 * F1 + 2.12 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 78.8 * L1
    ),
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.75,
    -(
        0.33 * B1 + 0.56 * C1 + 1.99 * D1 + 2.88 * E1 + 1.45 * F1 + 1.67 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ),
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.51,
    -(
        0.07 * B1 + 0.24 * C1 + 0.63 * D1 + 0.72 * E1 + 0.54 * F1 + 0.55 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ),
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.14,
    -(
        0.43 * B1 + 0.69 * C1 + 2.62 * D1 + 2.73 * E1 + 2.38 * F1 + 1.53 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ),
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.44,
    -(
        0.38 * B1 + 0.78 * C1 + 2.34 * D1 + 3.3 * E1 + 1.9 * F1 + 1.79 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1
    ) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.52,
    -(
        0.04 * B1 + 0.13 * C1 + 0.4 * D1 + 5.34 * E1 + 0.2 * F1 + 0.75 * G1 + 0 * H1 + 35 * I1 + 24 * J1 + 0 * K1 + 0 * L1
    ),
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.5,
    -(
        0.3 * B1 + 0.18 * C1 + 0.71 * D1 + 3.05 * E1 + 1.15 * F1 + 1.1 * G1 + 0 * H1 + 0 * I1 + 10 * J1 + 0 * K1 + 0 * L1
    ) /
    B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.45])

# # 对于S阶段的约束
pop.CV = np.hstack([
    # 用量约束, 所有成长阶段都要遵守

```



5.3.4. 对于变异阶段的约束条件


```

pop.CV = np.hstack([
    # 用量约束, 所有成长阶段都要遵守
    0.20 - (B1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)),
    (B1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.98,
    (C1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.20,
    0.10 - (D1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)),
    (D1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.60,
    (E1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.05,
    (F1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.09,
    (G1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.07,
    (H1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,
    (I1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.02,
    (J1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.10,
    (K1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,
    (L1 / (B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1)) - 0.03,
    -(
        3.85 * B1 + 3.23 * C1 + 4.05 * D1 + 4.25 * E1 + 3.4 * F1 + 3.44 * G1 + 7.7 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 3
    ),
    -(
        9.4 * B1 + 17.3 * C1 + 49.9 * D1 + 68.5 * E1 + 44.9 * F1 + 37.8 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 13.2,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 13.2,
    ),
    -(
        0.43 * B1 + 1.18 * C1 + 3.68 * D1 + 3.99 * E1 + 4.96 * F1 + 2.65 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 0.19,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.19,
    ),
    -(
        0.29 * B1 + 0.49 * C1 + 1.38 * D1 + 1.94 * E1 + 1.27 * F1 + 1.06 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 0.19,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.19,
    ),
    -(
        1.05 * B1 + 1.07 * C1 + 3.9 * D1 + 4.95 * E1 + 2.64 * F1 + 2.56 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 0.54,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.54,
    ),
    -(
        0.31 * B1 + 0.54 * C1 + 2.28 * D1 + 2.8 * E1 + 1.39 * F1 + 1.45 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 0.33,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.33,
    ),
    -(
        0.2 * B1 + 0.27 * C1 + 0.72 * D1 + 1.92 * E1 + 0.71 * F1 + 0.71 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 99 * K1 + 0 * L1 + 0.16,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.16,
    ),
    -(
        0.27 * B1 + 0.7 * C1 + 3.13 * D1 + 5.24 * E1 + 1.85 * F1 + 2.12 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 78.8 * L1 + 0.6,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.6,
    ),
    -(
        0.33 * B1 + 0.56 * C1 + 1.99 * D1 + 2.88 * E1 + 1.45 * F1 + 1.67 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 0.41,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.41,
    ),
    -(
        0.07 * B1 + 0.24 * C1 + 0.63 * D1 + 0.72 * E1 + 0.54 * F1 + 0.55 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 0.11,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.11,
    ),
    -(
        0.43 * B1 + 0.69 * C1 + 2.62 * D1 + 2.73 * E1 + 2.38 * F1 + 1.53 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 0.34,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.34,
    ),
    -(
        0.38 * B1 + 0.78 * C1 + 2.34 * D1 + 3.3 * E1 + 1.9 * F1 + 1.79 * G1 + 0 * H1 + 0 * I1 + 0 * J1 + 0 * K1 + 0 * L1 + 0.4,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.4,
    ),
    -(
        0.04 * B1 + 0.13 * C1 + 0.4 * D1 + 5.34 * E1 + 0.2 * F1 + 0.75 * G1 + 0 * H1 + 35 * I1 + 24 * J1 + 0 * K1 + 0 * L1 + 0.45,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.45,
    ),
    -(
        0.3 * B1 + 0.18 * C1 + 0.71 * D1 + 3.05 * E1 + 1.15 * F1 + 1.1 * G1 + 0 * H1 + 0 * I1 + 10 * J1 + 0 * K1 + 0 * L1 + 0.4,
        B1 + C1 + D1 + E1 + F1 + G1 + H1 + I1 + J1 + K1 + L1 + 0.4)
])

# 用法 (1 个动态)
if solve():
    problem = MyProblem() # 实例化问题对象

```

5.3.5. 进化函数

设置最大进化代数,差分进化中的 F,设置相应的交叉频率,以及绘制相应的效果图用来进行更显著的表达

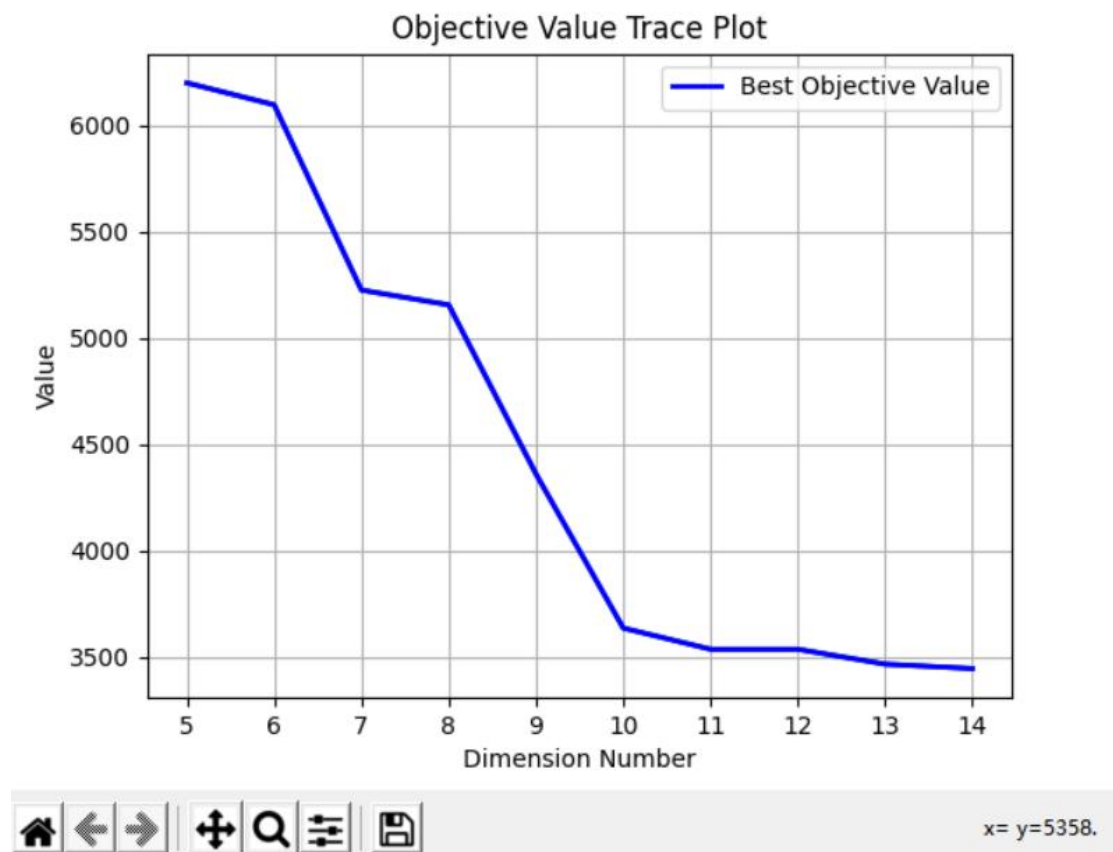
```

GeneticAlgorithm = ea.soea_DE_best_1_L_templet(problem, population)
# 最大进化代数
GeneticAlgorithm .MAXGEN = 1000
# 差分进化中的参数F
GeneticAlgorithm .mutOper.F = 0.5
# 设置交叉概率
GeneticAlgorithm .recOper.XOVR = 0.5
GeneticAlgorithm .logTras = 1
GeneticAlgorithm .verbose = True
# 设置绘图方式 (0: 不绘图; 1: 绘制结果图; 2: 绘制目标空间过程动画; 3: 绘制决策空间过程动画)
GeneticAlgorithm .drawing = 1
# 调用算法模板
[BestIndi, population, ] = GeneticAlgorithm .run()
BestIndi.save()

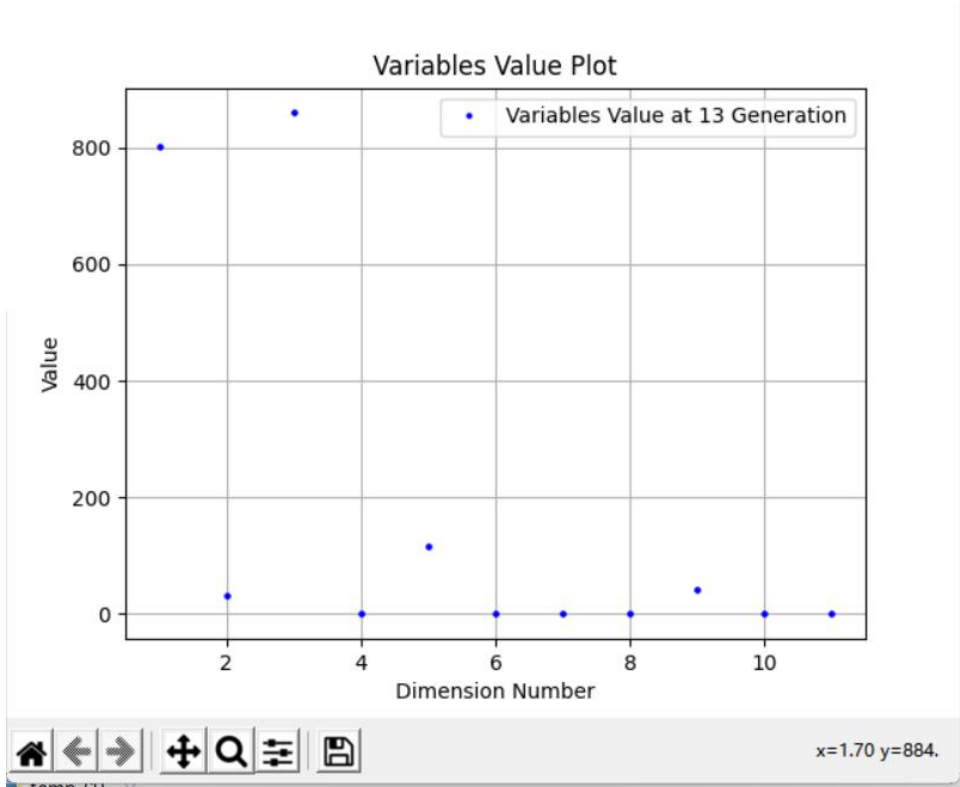
```

5.3.6. 决策过程展示

1) 目标空间过程动画



2) 决策空间过程动画



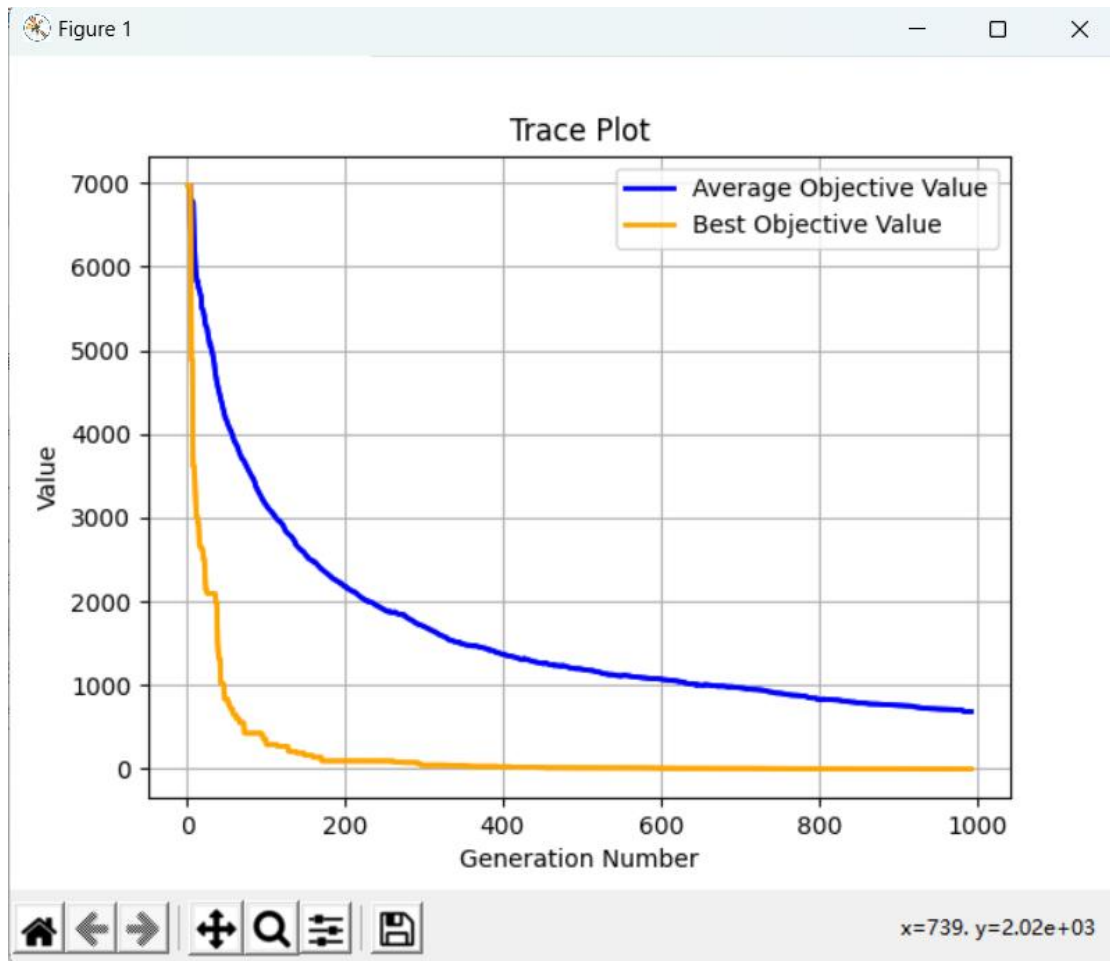
5. 4. 代码运行展示:

1. 对于 20-50kg 的生猪，每一千克需要的饲料结果如下:

迭代次数: 1000000
时间花费 1.6582903861999512 秒
最优的目标函数值为: 2.270594059105341
最优的控制变量值为:
最优用量:玉米:0.858 kg 麦麸:0.0 kg 豆粕:0.337 kg 鱼粉:0.0 kg
棉籽油:0.0 kg 菜籽油:0.0 kg 植物油:0.0 kg 石粉:0.0 kg
磷酸氢钙:0.029 kg 蛋氨酸:0.0 kg 赖氨酸:0.0 kg 复合预混料:0.03708978177618382 kg 食盐:0.003720364741641337 kg

原料种类	质量
玉米	0.858
麦麸	0.0
豆粕	0.337
鱼粉	0.0
棉籽油	0.0
菜籽油	0.0
植物油	0.0
石粉	0.0
磷酸氢钙	0.029
蛋氨酸	0.0

赖氨酸	0.0
复合预混料	0.03708978
食盐	0.00372036
最优目标值	1.658290



2. 对于 50-80kg 的生猪，每一千克需要的饲料结果如下：

迭代次数: 1000000

时间花费 1.4418842792510986 秒

最优的目标函数值为: 1.804491413543019

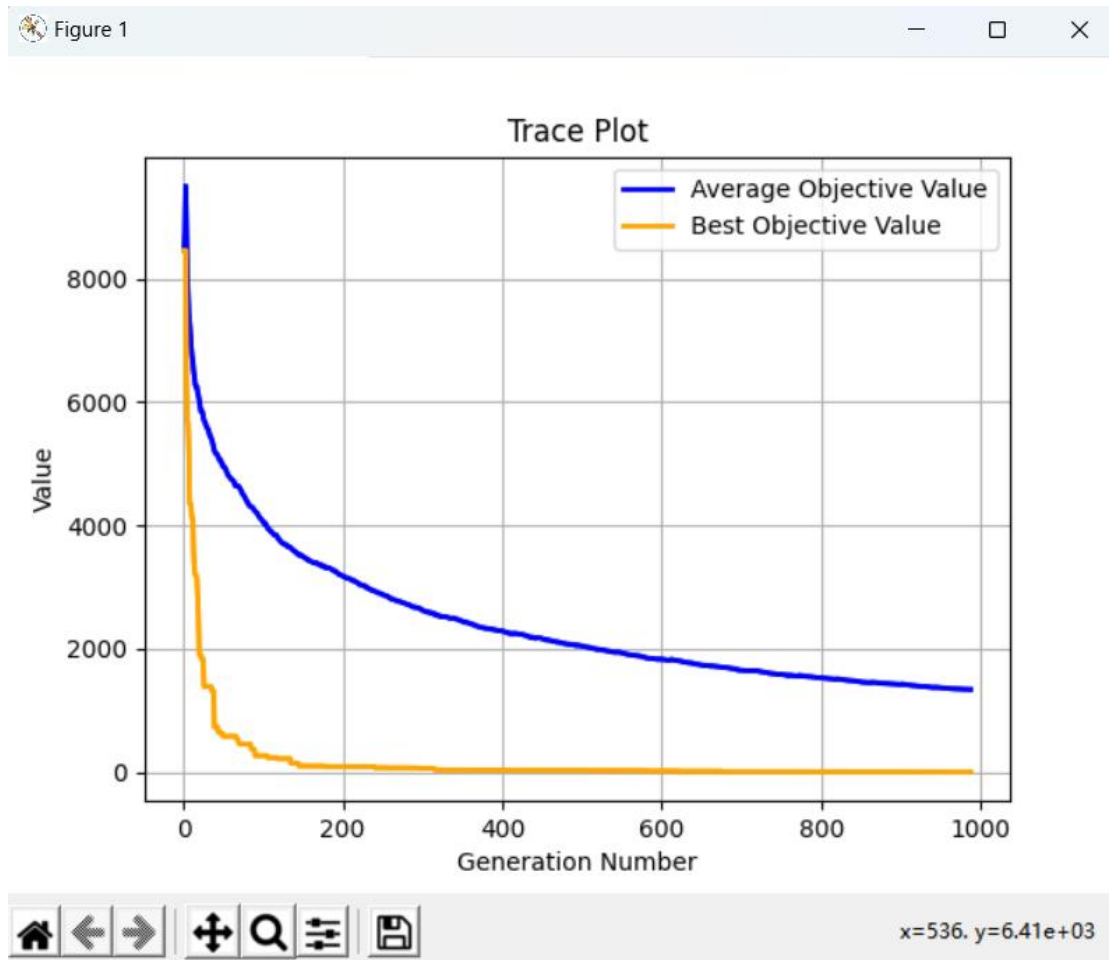
最优的控制变量值为:

最优用量: 玉米:0.547 kg 麦麸:0.0 kg 豆粕:0.35 kg 鱼粉:0.0 kg

棉籽油:0.0 kg 菜籽油:0.0 kg 植物油:0.0 kg 石粉:0.017 kg

磷酸氢钙:0.0 kg 蛋氨酸:0.0 kg 赖氨酸:0.0 kg 复合预混料:0.02769612789496079 kg 食盐:0.002778115501519757 kg

原料种类	质量
玉米	0.547
麦麸	0.0
豆粕	0.35
鱼粉	0.0
棉籽油	0.0
菜籽油	0.0
植物油	0.0
石粉	0.017
磷酸氢钙	0.0
蛋氨酸	0.0
赖氨酸	0.0
复合预混料	0.02769612
食盐	0.00277811
最优目标值	1.804491



3.对于 80-120kg 的生猪，每一千克需要的饲料结果如下：

迭代次数: 1000000

时间花费 1.50736665725708 秒

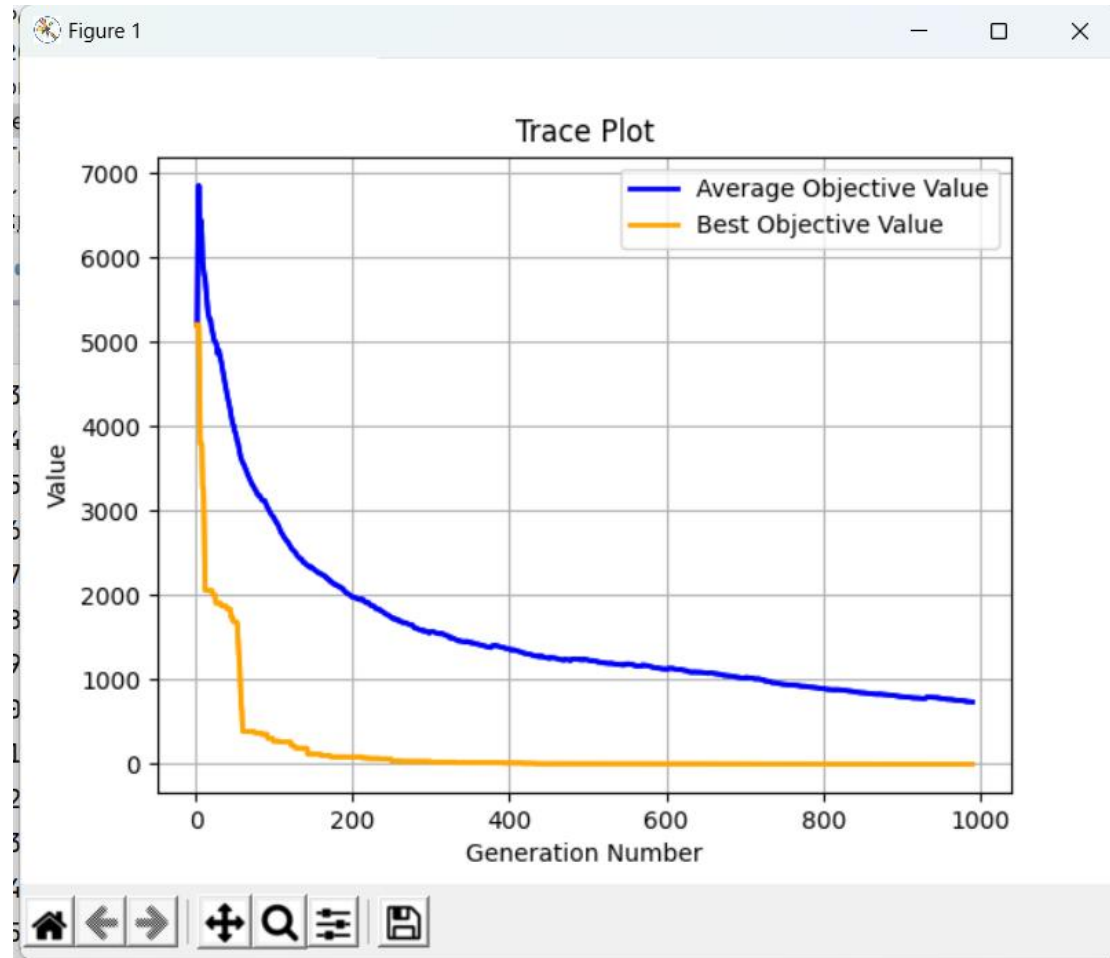
最优的目标函数值为: 3.2023018248537483

最优的控制变量值为:

最优用量:玉米:1.594 kg 麦麸:0.0 kg 豆粕:0.273 kg 鱼粉:0.0 kg

棉籽油:0.0 kg 菜籽油:0.0 kg 植物油:0.0 kg 石粉:0.0 kg

磷酸氢钙:0.035 kg 蛋氨酸:0.0 kg 赖氨酸:0.0 kg 复合预混料:0.0576346118776974 kg 食盐:0.005781155015197568 kg



原料种类	质量
玉米	1.594
麦麸	0.0
豆粕	0.273
鱼粉	0.0
棉籽油	0.0
菜籽油	0.0
植物油	0.0
石粉	0.0
磷酸氢钙	0.035
蛋氨酸	0.0
赖氨酸	0.0

复合预混料	0.05763461
食盐	0.00578115
最优目标值	1.507366

6.不同算法的性能分析

6.1. 数据对比

枚举

配方	20-50kg	50-80kg	80-120kg
玉米	1.1	0.4	0.3
麦麸	0.4	0.2	0.2
豆粕	0.3	1.0	0.8
鱼粉	0.1	0.0	0.0
棉籽油	0.0	0.0	0.1
菜籽油	0.1	0.1	0.1
植物油	0.1	0.1	0.1
石粉	0.0	0.0	0.0
磷酸氢钙	0.2	0.2	0.2
蛋氨酸	0.1	0.1	0.1
赖氨酸	0.1	0.1	0.1
复合预混料	0.02	0.017	0.01
食盐	0.006	0.00	0.005
最优目标值	9.907242	9.886155	9.315431

线性规划

配方	20-50kg	50-80kg	80-120kg
玉米	0.4681	0.54717	0.71538
麦麸	0.2	0.2	0.0
豆粕	0.1	0.1	0.1
鱼粉	0.0	0.0	0.0
棉籽油	0.09	0.00047	0.0
菜籽油	0.07	0.07	0.07
植物油	0.0	0.0	0.0
石粉	0.02	0.02	0.02
磷酸氢钙	0.03819	0.04936	0.08162
蛋氨酸	0.0	0.0	0.0
赖氨酸	0.00071	0.0	0.0
复合预混料	0.01	0.01	0.01
食盐	0.003	0.003	0.003

最优目标值	1.697845	1.672722	1.667836
遗传算法			
配方	20-50kg	50-80kg	80-120kg
玉米	0.858	0.547	1.594
麦麸	0.0	0.0	0.0
豆粕	0.337	0.35	0.273
鱼粉	0.0	0.0	0.0
棉籽油	0.0	0.0	0.0
菜籽油	0.0	0.0	0.0
植物油	0.0	0.0	0.0
石粉	0.0	0.017	0.0
磷酸氢钙	0.029	0.0	0.035
蛋氨酸	0.0	0.0	0.0
赖氨酸	0.0	0.0	0.0
复合预混料	0.03708978	0.02769612	0.05763461
食盐	0.00372036	0.00277811	0.00578115
最优目标值	1.658290	1.804491	1.507366

6.2. 模型评价

枚举搜索法：

运用枚举搜索尝试找到一种饲料原料的组合方案，使得该方案在满足生猪重量对应的营养标准的情况下，总价格最低。在每个原料的使用量上都进行了一定的划分，并通过循环穷举各种组合方案，并找到最优方案和对应的价格。此类型枚举搜索方法在问题规模小的情况下可以得到全局最优解，但是在问题规模扩大时会带来计算复杂度极高的问题。

线性规划：

线性规划提供了清晰的数学模型，便于建模和求解，并且针对线性规划问题有很多高效的求解算法可供使用，可以快速得到最优解。除此之外，可以用来解决多种约束条件下的优化问题，包括成本优化、

资源分配等。但是，线性规划问题对问题的假设和约束条件有严格的要求，某些实际问题可能无法完全符合线性规划的假设，而且要求所有变量是连续的，对于离散变量的优化问题不太适用，只能用于处理线性关系的问题，无法处理非线性关系的优化问题。综上所述，线性规划作为一种优化工具，可以有效地处理具有线性关系、连续变量的优化问题，但在面对非线性关系、离散变量或者强约束条件的问题时可能不太适用。

遗传算法：

通常遗传算法适用于解决非线性优化问题，对于复杂的、非凸的优化问题有较好的适应能力，而且求解过程可以并行化，能够利用计算资源并行求解，加快求解速度，还能够搜索解空间较广，有较好的全局搜索能力，能够找到较优的解决方案，能有效解决多目标优化问题，通过适当的调整参数和适应性函数，实现多目标的优化求解。但是遗传算法的收敛速度受到许多因素的影响，可能会出现局部最优解的情况而且易受各种参数设置的影响，因此需要进行大量的试验和调参。综上所述，遗传算法适用于复杂、非线性、多目标优化问题，具有较好的全局搜索能力。

7.参考文献

[1] 杨莎莎. 基于改进遗传算法的优化生猪饲料配方研究[D].安徽农业大学,2020.DOI:10.26919/d.cnki.gannu.2019.000601.

[2] 黄科. 遗传算法在猪饲料配方系统中的应用研究[D].苏州大学,2014.

[3] 张大伟.线性规划优化饲料配方的模型及其软件设计[J].中国饲料,2018(12):83-85.DOI:10.15906/j.cnki.cn11-2975/s.20181218.

[4] 张元跃,贺喜,刘国民等.基于线性规划和组合理论的饲料多配方平行设计的探讨[J].生物数学学报,2017,32(04):523-537.

[5] 曹志勇. 遗传算法在猪饲料配方设计中的应用研究[D].昆明理工大学,2007.

8.附录

饲料原料及其营养成分含量

原料名称	玉米	麦麸	豆粕	鱼粉	棉籽油	菜籽油	植物油	石粉	磷酸氢钙	蛋氨酸	赖氨酸	复合预混料	食盐
消化能 (Mcal/kg)	3.85	3.23	4.05	4.25	3.4	3.44	7.7	0	0	0	0	0	0
粗蛋白 (%)	9.4	17.3	49.9	68.5	44.9	37.8	0	0	0	0	0	0	0
精氨酸 (%)	0.43	1.18	3.68	3.99	4.96	2.65	0	0	0	0	0	0	0
组氨酸 (%)	0.29	0.49	1.38	1.94	1.27	1.06	0	0	0	0	0	0	0
亮氨酸 (%)	1.05	1.07	3.9	4.95	2.64	2.56	0	0	0	0	0	0	0
异亮氨酸 (%)	0.31	0.54	2.28	2.8	1.39	1.45	0	0	0	0	0	0	0
蛋氨酸 (%)	0.2	0.27	0.72	1.92	0.71	0.71	0	0	0	99	0	0	0
赖氨酸 (%)	0.27	0.7	3.13	5.24	1.85	2.12	0	0	0	0	78.8	0	0
苏氨酸 (%)	0.33	0.56	1.99	2.88	1.45	1.67	0	0	0	0	0	0	0
色氨酸 (%)	0.07	0.24	0.63	0.72	0.54	0.55	0	0	0	0	0	0	0
苯丙氨酸 (%)	0.43	0.69	2.62	2.73	2.38	1.53	0	0	0	0	0	0	0
缬氨酸 (%)	0.38	0.78	2.34	3.3	1.9	1.79	0	0	0	0	0	0	0
钙 (%)	0.04	0.13	0.4	5.34	0.2	0.75	0	35	24	0	0	0	0
总磷 (%)	0.3	0.18	0.71	3.05	1.15	1.1	0	0	10	0	0	0	0
用量上限 (%)	90	20	60	5	9	7	3	2	10	3	3	0	0
用量下限 (%)	20	0	10	0	0	0	0	0	0	0	0	0	0
等量使用 (%)	0	0	0	0	0	0	0	0	0	0	1	0.3	
价格 (元/kg)	1.5	1.5	2.8	6	1.6	1.4	6	0.2	1.35	28	22	10	0.86

生猪饲养标准

营养指标\体重阶段	20-50kg	50-80kg	80-120kg
消化能 (Mcal/kg)	3.4	3.4	3.4
粗蛋白质, %	18	15.5	13.2
精氨酸, %	0.37	0.27	0.19
组氨酸, %	0.3	0.24	0.19
亮氨酸, %	0.9	0.71	0.54
异亮氨酸, %	0.51	0.42	0.33
蛋氨酸, %	0.25	0.2	0.16
赖氨酸, %	0.95	0.75	0.6
苏氨酸, %	0.61	0.51	0.41
色氨酸, %	0.17	0.14	0.11
苯丙氨酸, %	0.55	0.44	0.34
缬氨酸, %	0.64	0.52	0.4
钙, %	0.6	0.5	0.45
总磷, %	0.5	0.45	0.4

9. 小组分工贡献:

李莎:负责线性规划 python 代码实现, 论文整体框架, 线性规划、枚举搜索论文撰写。参与遗传算法思路优化。

张高博:负责遗传算法优化代码实现, 枚举搜索 python 代码实现, 遗传算法论文撰写。参与线性规划思路优化。

孙华斌:负责遗传算法 python 代码实现。参与遗传算法思路优化。

王绮文:负责枚举搜索算法分析, 算法性能分析, 表格处理论文撰写。参与线性规划算法思路优化。